

DBMS – PAYMENT MANAGEMENT SYSTEM

Meesho Jewelry Marketplace Database

1. Objectives

1. Create a **PAYMENT** table.
2. Store payment details such as payment mode, date, amount, and status.
3. Manage successful, failed, and pending transactions.
4. Analyze payment methods used by customers.
5. Generate payment transaction reports.

2. Database Design

The **PAYMENT** table is designed to store payment information related to customer orders.

Column Name	Data Type	Description
PAYMENT_ID	NUMBER(5)	Unique ID for each payment
ORDER_ID	NUMBER(5)	Identifies the related order
PAYMENT_MODE	VARCHAR2(30)	Mode used for payment
PAYMENT_DATE	DATE	Date of payment
PAYMENT_AMOUNT	NUMBER(10,2)	Amount paid
PAYMENT_STATUS	VARCHAR2(30)	Status of the payment

3. Relationship

The **PAYMENT** table is connected to the **ORDERS** table using **ORDER_ID**.

Relationship:

ORDERS 1 : 1 PAYMENT

- ORDERS.ORDER_ID → Primary Key
- PAYMENT.ORDER_ID → Foreign Key

This relationship connects each payment transaction with its corresponding order.

4. Table Creation

```
SQL> CREATE TABLE PAYMENT (  
  2     PAYMENT_ID NUMBER(5) PRIMARY KEY,  
  3     ORDER_ID NUMBER(5),  
  4     PAYMENT_MODE VARCHAR2(30) NOT NULL,  
  5     PAYMENT_DATE DATE NOT NULL,  
  6     PAYMENT_AMOUNT NUMBER(10,2) NOT NULL,  
  7     PAYMENT_STATUS VARCHAR2(30) NOT NULL,  
  8     CONSTRAINT FK_PAYMENT_ORDER  
  9         FOREIGN KEY (ORDER_ID)  
 10         REFERENCES ORDERS(ORDER_ID)  
 11 );
```

Table created.

5. Data Insertion

Six payment records were inserted based on the existing orders.

```
SQL> INSERT INTO PAYMENT VALUES  
  2  (1, 1001, 'UPI', DATE '2026-09-22', 1198.00, 'Successful');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO PAYMENT VALUES  
  2  (2, 1002, 'Card', DATE '2026-09-22', 999.00, 'Successful');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO PAYMENT VALUES  
  2  (3, 1003, 'UPI', DATE '2026-09-23', 599.00, 'Successful');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO PAYMENT VALUES  
  2  (4, 1004, 'Cash on Delivery', DATE '2026-09-23', 449.00, 'Successful');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO PAYMENT VALUES  
  2  (5, 1005, 'Card', DATE '2026-09-24', 349.00, 'Failed');  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO PAYMENT VALUES  
  2  (6, 1006, 'UPI', DATE '2026-09-24', 799.00, 'Pending');  
  
1 row created.
```

6. Managing Payment Transactions

Successful Transactions

This query displays all successfully completed payment transactions.

```
SQL> SELECT *
2 FROM PAYMENT
3 WHERE PAYMENT_STATUS = 'Successful';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS
1	1001	UPI	22-SEP-26	1198	Successful
2	1002	Card	22-SEP-26	999	Successful
3	1003	UPI	23-SEP-26	599	Successful
4	1004	Cash on Delivery	23-SEP-26	449	Successful

Failed Transactions

This query displays payment transactions that were unsuccessful.

```
SQL> SELECT *
2 FROM PAYMENT
3 WHERE PAYMENT_STATUS = 'Failed';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS
5	1005	Card	24-SEP-26	349	Failed

Pending Transactions

This query displays transactions that are still pending.

```
SQL> SELECT *
2 FROM PAYMENT
3 WHERE PAYMENT_STATUS = 'Pending';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS
6	1006	UPI	24-SEP-26	799	Pending

7. Analyzing Payment Methods

The number of transactions made using each payment method can be analyzed using:

```
SQL> SELECT PAYMENT_MODE, COUNT(*) AS TOTAL_TRANSACTIONS
2 FROM PAYMENT
3 GROUP BY PAYMENT_MODE;
```

PAYMENT_MODE	TOTAL_TRANSACTIONS
UPI	3
Card	2
Cash on Delivery	1

This helps identify the different payment methods used by customers and the number of transactions made through each method.

Total Amount by Payment Method

```
SQL> SELECT PAYMENT_MODE, SUM(PAYMENT_AMOUNT) AS TOTAL_AMOUNT
2 FROM PAYMENT
3 GROUP BY PAYMENT_MODE;
```

PAYMENT_MODE	TOTAL_AMOUNT
UPI	2596
Card	1348
Cash on Delivery	449

This query calculates the total payment amount processed through each payment method.

8. Payment Transaction Report

A complete payment transaction report can be generated by joining the PAYMENT and ORDERS tables.

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS	ORDER_STATUS
1	1001	UPI	22-SEP-26	1198	Successful	Shipped
2	1002	Card	22-SEP-26	999	Successful	Packed
3	1003	UPI	23-SEP-26	599	Successful	Shipped
4	1004	Cash on Delivery	23-SEP-26	449	Successful	Delivered
5	1005	Card	24-SEP-26	349	Failed	Pending
6	1006	UPI	24-SEP-26	799	Pending	Pending

6 rows selected.